Assignment 5: Optimising Attention on a CPU

CS6886: Systems for Deep Learning, IIT Madras

Due: November 6, 2026

Task

You are given a naive C implementation of single-head attention,

$$O = \text{softmax}\left(\frac{QK^\top}{\sqrt{d}}\right)V,$$

where Q, K, V and O are $N \times d$ row-major FP32 matrices and the softmax is applied to each row. Make it fast on your own machine, step by step, and explain every result with measurements.

Starter code

Create your own repository from the starter code:

`https://github.com/NMJ0/Assignment-5-Systems-for-Deep-learning`

Questions

Q1. Your machine **(20 marks)**

Fill in `machine_spec.txt`: CPU model, cores (physical, logical, performance and efficiency), clock frequencies, SIMD width and FMA units, cache sizes, memory type and size. From these, compute the peak FP32 GFLOP/s (one core and all cores), the memory bandwidth and the ridge point. Show your calculations in the report.

Answer

All runs are on an **Intel Core Ultra 7 165H** (Meteor Lake), compiled with clang 23.1.5 at `-O3 -march=native`. Every field is in `machine_spec.txt`; these are the ones the calculations below depend on:

- 16 cores / 22 threads: 6 P-cores (Redwood Cove, 2 threads each), 8 E-cores and 2 low-power E-cores (Crestmont).
- P-core 1.4 GHz base, 5.0 GHz max; E-core 3.8 GHz max; LP E-core 2.5 GHz max.
- AVX2 + FMA, 256-bit vectors (8 floats), 16 YMM registers, no AVX-512. A P-core has two 256-bit FMA units; an E-core has two 128-bit ones.
- L1D 48 KB (P) / 32 KB (E), L2 2 MB per P-core and 2 MB per 4-E-core cluster, L3 24 MB shared. 64-byte lines.
- 64 GB LPDDR5x-7467 on a 128-bit bus (8 x 16-bit channels).

Peak FP32. One FMA is 2 FLOPs, so a P-core does 2 units x 8 lanes x 2 = **32 FLOP/cycle** and an E-core 2 x 4 x 2 = 16 FLOP/cycle.

- One P-core: 32 x 5.0 GHz = **160 GFLOP/s**.
- All cores: 6 x 32 x 5.0 + 8 x 16 x 3.8 + 2 x 16 x 2.5 = 960 + 486 + 80 = **1526 GFLOP/s**. This is an upper bound on paper; in practice the chip can't hold every core at max turbo at the same time.

Memory bandwidth. 7467 MT/s x 16 bytes per transfer = **119.5 GB/s**.

Ridge point = peak / bandwidth: 160 / 119.5 = **1.34 FLOP/B** for one core and 1526 / 119.5 = **12.8 FLOP/B** for all cores.

Measured peaks. Spec-sheet numbers are optimistic, so I measured both as well: an FMA loop with 12 independent accumulators (enough to hide the 4-cycle FMA latency) and a triad `y = y + a*x` over two 512 MB arrays.

	1 thread	6	12	16	22
FMA (GFLOP/s)	70.9	361.8	462.8	500.2	491.0
triad (GB/s)	28.8	68.4	79.3	77.9	80.7

70.9 GFLOP/s on one core works out to an effective clock of 70.9 / 32 = 2.2 GHz (about 1.9 GHz with 6 cores busy), well below the 5 GHz turbo once the load is sustained. Bandwidth reaches 68 % of the theoretical figure. The measured ridge points are 70.9 / 28.8 = 2.5 FLOP/B for one core and 500 / 80.7 = 6.2 FLOP/B for all cores. In the later questions I give % of peak against both 160 (theory) and 70.9 (measured), because the measured number is the fairer target.

Q2. Naive baseline (20 marks)

Run `make naive`. For each input, report the time, GFLOP/s and percentage of the peak. How does the time change with N and with d? Explain where the time goes.

Answer

N	d	time (ms)	GFLOP/s	% of 160	% of 70.9
1024	64	171.8	1.56	1.0%	2.2%
2048	64	721.0	1.49	0.9%	2.1%
4096	64	2860.9	1.50	0.9%	2.1%
4096	128	6940.5	1.24	0.8%	1.7%

Scaling. Time goes up with N^2: doubling N costs 4.2x (1024 to 2048) and 4.0x (2048 to 4096), which is just the $4N^2d$ FLOP count. Doubling d at N = 4096 costs 2.43x, a bit more than the 2x the work grows by, so GFLOP/s drops from 1.50 to 1.24.

Where the time goes. Timing the three steps separately gave QK^T 29 %, softmax 4 %, S·V 67 % at N = 4096, d = 64, and nearly the same split at N = 1024.

- **S·V (two thirds).** The inner loop walks down a column of V, so consecutive iterations are d x 4 = 256 bytes apart and each multiply touches a new cache line. It is also one long serial sum (`acc += ...`), so each FMA waits about 4 cycles for the previous one.
- **QK^T (a third).** Memory access is fine (both rows are contiguous), but it is the same serial dot product. Clang will not vectorise a float sum without `-ffast-math`, because reordering the additions changes the result. One scalar FMA every ~4 cycles at ~2.2 GHz is about 1 GFLOP/s, which is close to what I see.
- **Softmax (small).** N^2 scalar `expf` calls; each row (16 KB) sits in L1 across its three passes.

DRAM is not the problem even though S is 64 MB at N = 4096 and does not fit in L3: the naive code moves roughly $16N^2$ bytes in 2.9 s, about 0.1 GB/s. It is under 1 % of peak because it runs one scalar, dependent FMA at a time. The d = 128 case is worse because V grows to 2 MB, no longer fits a P-core's L2, and the column stride doubles to 512 bytes.

Q3. Optimisations (20 marks)

Write each variant, run it, and report the same measurements as Q2 plus the speedup over the naive variant. `make check` must pass. Explain what each change does and what limits performance afterwards. Do not use BLAS or other math libraries.

(a) `attn_loops.c`: reorder the loops so the compiler can vectorise them.

(b) `attn_tiled.c`: cache and register tiling.

(c) `attn_fused.c`: fuse the three steps row by row so the $N \times N$ matrix is never stored.

(d) `attn_best.c`: anything you want. Get the maximum single-threaded speedup you can, and explain what you did and why it works.

Answer

All variants pass `make check` on every input (max relative error 5e-7 to 6e-6, against the 1e-4 limit). % of peak is given against both the theoretical 160 and the measured 70.9 GFLOP/s single-core peak.

(a) attn_loops.c.

N	d	time (ms)	GFLOP/s	% of 160	% of 70.9	speedup
1024	64	20.1	13.36	8.3%	18.8%	8.6x
2048	64	87.4	12.29	7.7%	17.3%	8.2x
4096	64	322.7	13.31	8.3%	18.8%	8.9x
4096	128	537.6	15.98	10.0%	22.5%	12.9x

I transpose K once and switch step 1 to i-k-j order, so the inner loop becomes `s[j] += q[i][k] * Kt[k][j]`. Step 3 becomes i-j-k: `o[k] += p[j] * V[j][k]`. Both inner loops are now unit-stride with independent iterations, so clang emits 8-wide AVX2 FMAs without needing `-ffast-math`, and the strided column walk through V is gone. Softmax divides by multiplying with 1/sum. That is a 8.3-12.9x gain.

What limits it now: each vector FMA now needs two loads and a store (`s[j]` or `o[k]` is read and written every time), so the load/store ports run out before the FMA units. The N^2 `expf` calls are still scalar, and S still goes through memory.

(b) attn_tiled.c.

N	d	time (ms)	GFLOP/s	% of 160	% of 70.9	speedup
1024	64	13.9	19.34	12.1%	27.3%	12.4x
2048	64	65.9	16.30	10.2%	23.0%	10.9x
4096	64	254.9	16.85	10.5%	23.8%	11.2x
4096	128	377.6	22.75	14.2%	32.1%	18.4x

Register tiling: both products are computed in 4 x 16 blocks of the output, which is eight YMM accumulators that stay in registers for the whole k (or j) loop. Per step that is 2 vector loads of K^T plus 4 broadcasts of Q feeding 8 FMAs, instead of 2 loads + 1 store per FMA in (a). **Cache tiling:** in step 1 the outer loop walks K^T in tiles of d x 8192/d columns (32 KB), so one tile stays in L1 while all of Q streams past it. That makes it 11-18x faster than naive and 1.3-1.4x faster than (a). It is still plain C; the fixed-size `acc[4][16]` array is small enough for clang to unroll fully and keep in registers.

What limits it now: mostly the softmax. In the naive profile softmax was about 4 % of 2.86 s, so roughly 115 ms at N = 4096, d = 64. Tiled still runs that same scalar `expf` loop, so about 40 % of its 255 ms now goes on exponentials rather than matrix products. Writing and re-reading the 64 MB S matrix is the other cost.

(c) attn_fused.c.

N	d	time (ms)	GFLOP/s	% of 160	% of 70.9	speedup
1024	64	21.0	12.77	8.0%	18.0%	8.2x
2048	64	105.0	10.23	6.4%	14.4%	6.9x
4096	64	330.3	13.00	8.1%	18.3%	8.7x
4096	128	662.0	12.98	8.1%	18.3%	10.5x

For each query row I compute its N scores into one N-float buffer (16 KB, fits in L1), take the max, exponentiate, fold the row straight into O[i], and scale by 1/sum at the end. The N x N matrix never exists, so DRAM traffic drops from about $16N^2$ bytes (64+ MB) to the compulsory 16Nd (4 MB).

Fusing on its own did not make it faster: it is about as fast as (a) and slower than (b). That makes sense from the roofline (Q5): (a) was never memory-bound, so removing memory traffic has nothing to win. Fused still runs (a)'s axpy loops and scalar `expf`, and handling one row at a time means K^T and V (1 MB each) are re-read from L2 for every single row, with no reuse across rows. What fusion does give is a small working set, and (d) builds on that.

(d) attn_best.c.

N	d	time (ms)	GFLOP/s	% of 160	% of 70.9	speedup
1024	64	6.0	44.74	28.0%	63.1%	28.6x
2048	64	25.8	41.67	26.0%	58.8%	28.0x
4096	64	96.1	44.69	27.9%	63.0%	29.8x
4096	128	263.1	32.65	20.4%	46.1%	26.4x

Best is fused attention done four query rows at a time with AVX2 intrinsics. It is **26-30x faster than naive**, reaching 45 GFLOP/s, which is 63 % of the measured single-core peak. The pieces:

- **Pack K into 16-key panels** (`Kp[j/16][k][16]`), so the score kernel reads one contiguous 4 KB panel instead of hopping across d rows of K^T.
- **4 x 16 register tile for both products**: 8 FMAs per 2 loads + 4 broadcasts. Doing 4 rows at once also means every K and V value loaded from L2 gets used 4 times, not once as in (c).
- **Vector exp.** I pre-scale Q by $\log_2(e)/\sqrt{d}$ so the softmax needs 2^x, and compute it 8 at a time: split x into integer n and fraction f, evaluate a degree-6 polynomial for 2^f, and add n straight into the float's exponent bits. That is about 10 instructions per 8 values instead of 8 library calls, with relative error around 1e-7. This one change removes the bottleneck left by (b).
- **Vectorised max and sum** per row, and 1/sum folded into the final store of O.

What limits it now: the FMA units are busy but not full; the broadcasts and loads compete for issue slots, and I only use 8 of the 16 YMM registers as accumulators. The 4-row score buffer (64 KB) does not fit in the 48 KB L1, so the P·V loop re-reads it from L2 d/16 times. At d = 128 V becomes 2 MB, the whole L2, which is why that case drops to 33 GFLOP/s. The next steps would be online (flash-attention-style) softmax over key blocks, which keeps the scores in L1, and a 6 x 16 tile.

Q4. OpenMP (20 marks)

Speed up your best variant with OpenMP in `attn_omp.c`. Run it with different thread counts and report the time, speedup and parallel efficiency. Explain the scaling you see.

Answer

`attn_omp.c` is `attn_best.c` with one parallel region. K packing and Q scaling are split with `omp for`, and the 4-row blocks with `omp for schedule(dynamic, 4)`. Each thread has its own score buffer; blocks write disjoint rows of O, so there are no locks or reductions. I used dynamic scheduling because the cores are not equal: a P-core finishes a block about twice as fast as an E-core, and a static split would leave the P-cores idle at the barrier. Speedup and efficiency are against the 1-thread run of the same binary; all four inputs are in `results.csv`.

N = 4096, d = 64

threads	time (ms)	GFLOP/s	speedup	efficiency	vs naive
1	94.1	45.7	1.00x	100%	30x
2	47.9	89.6	1.96x	98%	60x
4	27.5	156.2	3.42x	85%	104x
6	20.8	206.6	4.52x	75%	138x
8	20.6	208.6	4.57x	57%	139x
12	18.4	233.4	5.11x	43%	155x
16	22.8	188.2	4.12x	26%	125x
22	22.0	195.4	4.28x	19%	130x

N = 4096, d = 128

threads	time (ms)	GFLOP/s	speedup	efficiency	vs naive
1	250.1	34.4	1.00x	100%	28x
2	113.1	76.0	2.21x	111%	61x
4	61.5	139.7	4.07x	102%	113x
6	50.1	171.6	5.00x	83%	139x
8	40.5	212.3	6.18x	77%	172x
12	36.1	237.7	6.92x	58%	192x
16	43.0	199.7	5.81x	36%	161x
22	40.1	214.1	6.23x	28%	173x

Why it scales like this. On this chip the threads are not equal, and the table shows it:

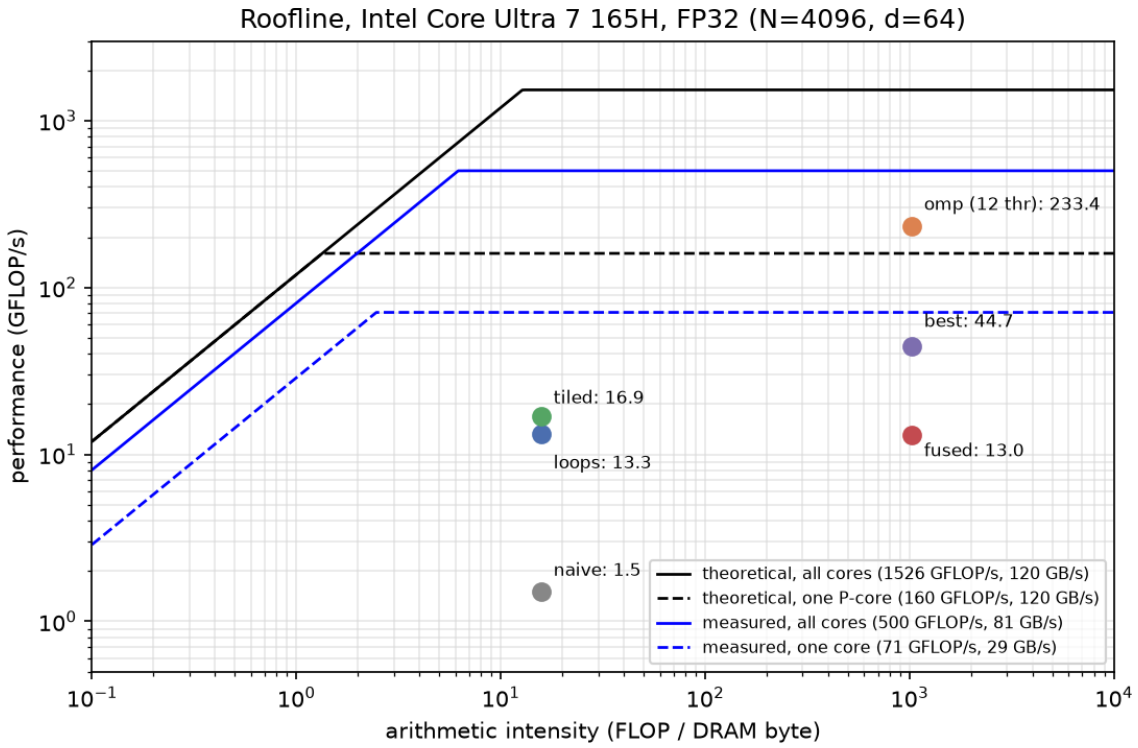
- **1-6 threads:** close to linear at 2 threads, then tapering. These threads land on the P-cores, but the more cores are busy the lower the clock goes (the FMA test gave 70.9 GFLOP/s on one core but only 60 per core with 6 busy), because all cores share one power budget. 6 threads gives about 4.5x.
- **8-12 threads:** small gains. The extra threads are either the second hyper-thread of a P-core, which shares the same two FMA units and so adds almost nothing to an FMA-bound kernel, or E-cores, which have half the vector width. 12 threads was the best setting: 5.1x at d = 64 and 6.9x at d = 128, about 233 GFLOP/s (47 % of the measured 500 GFLOP/s all-core peak).
- **16-22 threads: slower again.** This brings in the two LP E-cores, which are on a different tile, outside the L3, and slow. It also means more cores sharing the same power budget, so the P-cores clock down further.

Memory bandwidth is not the limit: at 233 GFLOP/s and ~1000 FLOP/B the kernel needs about 0.2 GB/s from DRAM. The small input scales worst (N = 1024 flattens at ~2.3 ms) because it only has 256 blocks and the fork/join and K packing are a visible share of 2 ms. The d = 128, 2-thread run came out slightly above 2x, which is within run-to-run variation from clock changes rather than a real superlinear effect.

Q5. Roofline (20 marks)

Plot the roofline of your machine using your Q1 numbers, and place every variant on it at $N = 4096$, $d = 64$. Explain how you computed each variant's arithmetic intensity and what limits each variant.

Answer



The plot has two pairs of roofs: theoretical (black: 160 / 1526 GFLOP/s, 119.5 GB/s) and measured (blue: 70.9 / 500 GFLOP/s, 28.8 / 80.7 GB/s). The points are the measured GFLOP/s of each variant at N = 4096, d = 64; omp is plotted at its best setting, 12 threads.

How I computed arithmetic intensity. FLOPs are the driver's count, $4N^2d$ ($2N^2d$ for QK^T plus $2N^2d$ for PV; exp and max are not counted), which is 1.07 GFLOP here. Bytes are the DRAM traffic each variant has to generate, worked out from its loop structure and the cache sizes:

- **Compulsory:** read Q, K, V and write O: 4 x N x d x 4 B = 16Nd = 4 MB.
- **naive, loops, tiled** also store S, which is 64 MB and does not fit in the 24 MB L3. It is written in step 1, read and written once in softmax (one 16 KB row stays in L1 across the three passes), and read once in step 3: 4 x $4N^2$ = $16N^2$ bytes. AI = $4N^2d$ / ($16N^2$ + 16Nd) = **15.8 FLOP/B**.
- **fused, best, omp** never store S, and K and V (1 MB each) stay in L2/L3, so only the compulsory traffic is left: AI = $4N^2d$ / 16Nd = N/4 = **1024 FLOP/B**.

What limits each one. Every point is to the right of every ridge point (at most 12.8 FLOP/B), so **none of the variants is memory-bound**. The distance from each point up to its roof is lost inside the core:

- **naive** (1.5 GFLOP/s, 2 % of the measured roof): scalar, latency-bound dependency chains and the strided column walk through V.
- **loops** (13.3) and **tiled** (16.9): vectorised, but limited by load/store port pressure (loops) and by the scalar `expf` calls (both). They sit at the same AI because tiling changes cache reuse, not DRAM traffic.
- **fused** (13.0): fusion moves the point far to the right but not up. It cuts memory traffic, which was not the bottleneck, and keeps the same instructions as loops.
- **best** (44.7, 63 % of the measured one-core roof): limited by FMA issue rate, with the loads and broadcasts competing for issue slots, and by L2 re-reads of the score buffer.
- **omp** (233, 47 % of the measured all-core roof): limited by the hybrid cores (hyper-threads share FMA units, E-cores are half-width) and by the all-core clock being lower than the single-core clock. With 1024 FLOP/B there is plenty of headroom before memory bandwidth matters.

These AI values are analytical. Real traffic is a little higher (for example, the read-for-ownership on stores to S), which would move the first three points slightly left, still well right of the ridge.

Submission

Submit a link to a **public GitHub repository** with exactly this structure:

Path	Contents
`src/`	`attn_naive.c` and all your variants
`input/`	The given inputs
`machine_spec.txt`	Fill in the empty fields
`results.csv`	Fill in the empty fields (add rows for the thread counts you tried)
`plots/roofline.png`	The roofline with all variants
`plots/time_N<N>_d<d>.png`	One plot per input: time of every variant
`Makefile`	Makefile
`assignment_handout.pdf`	This handout
`report.pdf`	Your answers to Q1–Q5

Notes on the submission

The repository follows the structure above. `make all` builds and times every variant and `make check` passes for all of them. The only Makefile change is `CC = clang -pthread`, which some toolchains need to link `clock_gettime`. Per-input time plots are in `plots/time_N<N>_d<d>.png`, for example:

